University of Tehran

Signals & Systems first computer assignment

Fourier Transform

Author
Ermia Abbasi

Course Instructor
Prof. Rabiei

Date
June 14, 2026

Contents

Abstract

This project uses MATLAB and Mathematica to simulate some problems related to Fourier Transform. from filtering the signals to handling the Moire effect using the Nyquist theorem.

1 Introduction

This project is deals with some of the basic signal processing techniques we learned during the signals and systems course. we begin with testing some of the Mathematica's built-in functions to plot some piece-wise linear signals. Then we take the Fourier Transform of a UnitBox and we filter it. We will see the effect of bandwidth of filtering on the output signal. Then we verify the parseval's theorem. After that we see the effect of time scaling on the Transformation. Now, we are ready to simulate the moire effect in MATLAB for a picture and solve that problem using the Gaussian filter. Finally, we see the effect of Nyquist theorem on a typical signal.

2 Problem 1: (Using Mathematica to plot important signals)

The piece-wise continuous signals can be written as a sum of unit steps and ramp signals. therefore, the representation of $x(t)$ and $y(t)$ is as follows:

$$x(t) = -2 + 2u(t+2) - r(t+2) + r(t+1) + r(t) - 2r(t-2)$$
$$y(t) = -1 + u(t+2) + r(t+2) - r(t) - 4u(t-1) + 2r(t-1) - 2r(t-2)$$

Now we plot these signals in Mathematica.

```
x[t_] := -2 + 2 UnitStep[t + 2] - Ramp[t + 2] + Ramp[t + 1] + Ramp[t] - 2 Ramp[t - 2];
Plot[x[t], {t, -3, 3}]
```

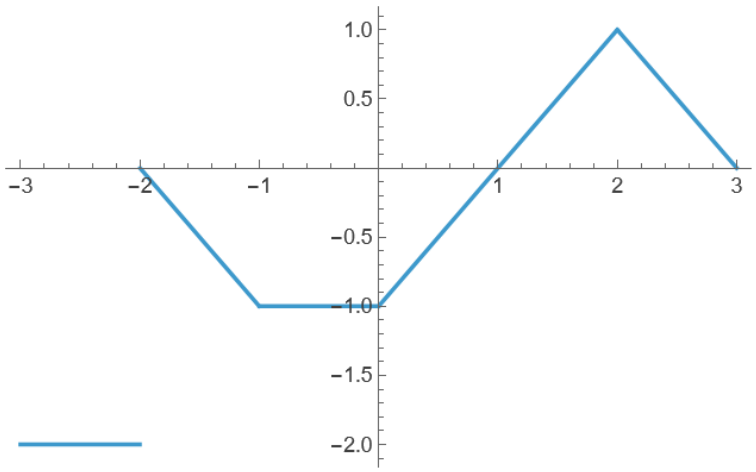

Figure 1. $x(t)$ signal

```
y[t_] = −1 + UnitStep[t + 2] + Ramp[t + 2] − Ramp[t] − 4 UnitStep[t − 1] + 2 Ramp[t − 1]−2
    Ramp[t − 2];
Plot[y[t], {t, −3, 3}]
```

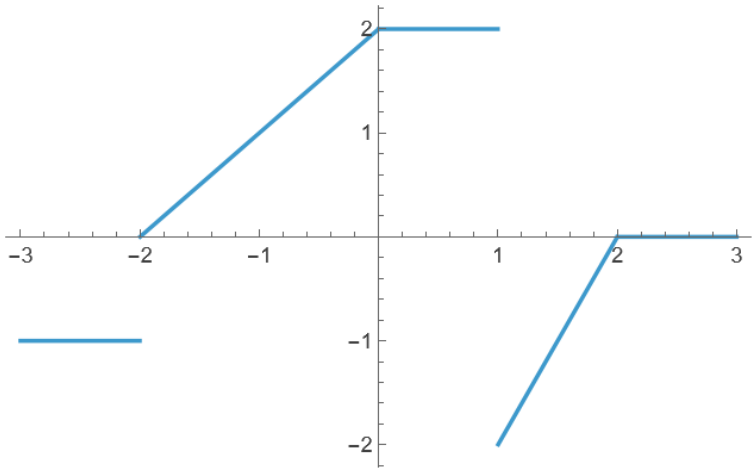

Figure 2. $y(t)$ signal

Now, we want to plot these signals:

$$z_1(t) = (x(2t) + y(t+1))\Pi(t/4)$$
$$z_2(t) = y(2t-1)\delta(t^2+3t-4)$$

```
z1[t_] := (x[2 t] + y[t + 1]) UnitBox[t/4];
Plot[{z1[t]}, {t, −3, 3}]
```

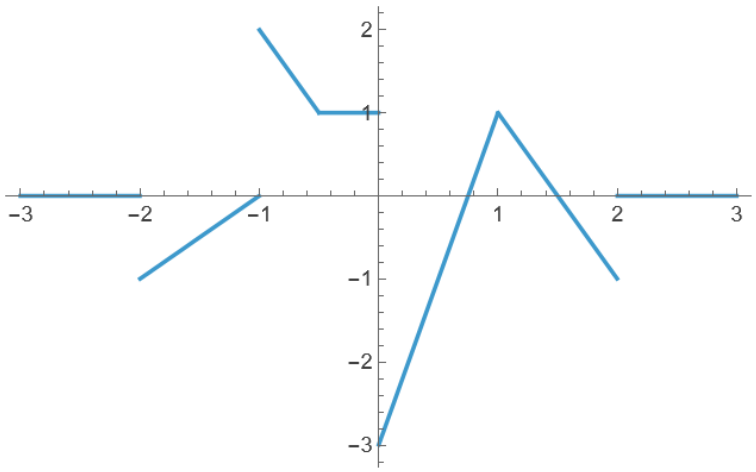

Figure 3. $z_1(t)$ signal

```
z2[t_] := y[2 t - 1] DiracDelta[t^2 + 3 t - 4];
Plot[{z2[t]}, {t, -3, 3}]
```

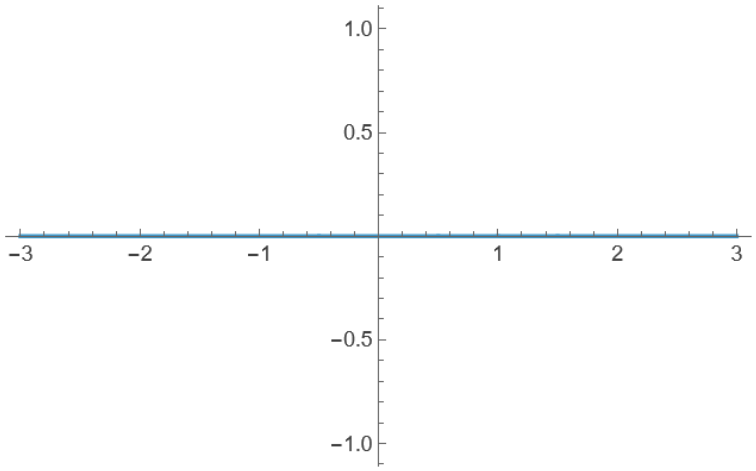

Figure 4. $z_2(t)$ signal

as seen in Figure 5, the Plot function doesn't show the discrete discontinuities of Dirac delta. but Its obvious that they are at $t = 1$ and $t = -4$.

3 Problem 2: (Introduction to Fourier transform and its properties)

The UnitBox in Mathematica is not 0.5 at its discontinuities. Therefore $f(t)$ is exactly the same as UnitBox. Therefore, we have these codes for this part.

```
f[t_] := UnitBox[t];
F[f_] = FourierTransform[f[t], t, f, FourierParameters -> {0, -2*Pi}]
Plot[{Abs[F[f]]}, {f, -20, 20}, PlotRange -> {0, 1.1}]
```

I used the PlotRange to see the peak at $x = 0$. when the second line is compiled it says the Fourier transform is parametrically $\text{sinc}(f)$.

Now we plot the signal in Mathematica.

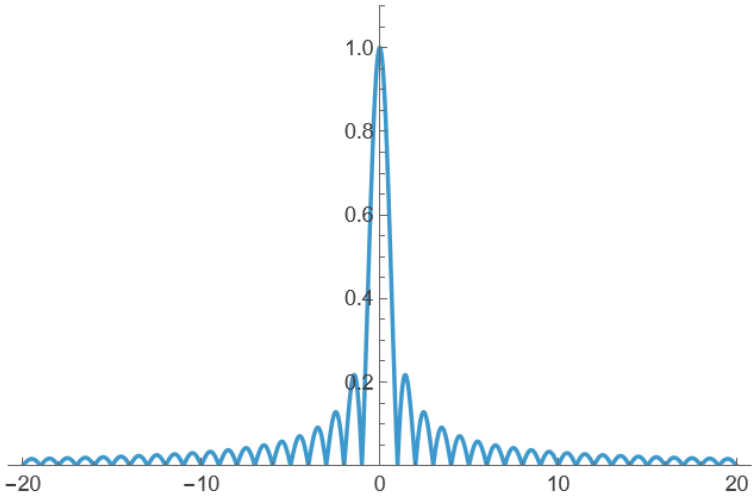

Figure 5. $F(f)$ signal

The FourierParameters that we used in our code are two parameters a and b. They are present in the main formula that Mathematica uses, which is as follows.

$$F(\omega) = \sqrt{\frac{|b|}{(2\pi)^{1-a}}} \int_{-\infty}^{\infty} f(t) e^{ib\omega t} dt$$

when we use $\{0, -2\pi\}$ we are actually using the Hz frequency not the angular one. This gives the same formula we had in the Signals and Systems course. In this convention, we don't need to worry about the division by $\sqrt{2\pi}$.

Now we're going to filter the signal. I use 1 Hz, 5 Hz and 20 Hz for f_0. we code these like this:

```
f0 = 20;
fFiltered[t_] = Integrate[F[f]*Exp[I*2Pi*f*t],{f, -f0, f0}];
f2 = 1;
fFiltered2[t_] = Integrate[F[f]*Exp[I*2 Pi*f*t], {f, -f2, f2}];
```

```
f3 = 5;
fFiltered3[t_] = Integrate[F[f]*Exp[I*2 Pi*f*t], {f, -f3, f3}];
Plot[{fFiltered3[t], fFiltered2[t], fFiltered[t], f[t]}, {t, -1, 1}, PlotLegends -> {5, 1,
    20, Main Signal}]
```

The graph that represent all 4 signals is as follows.

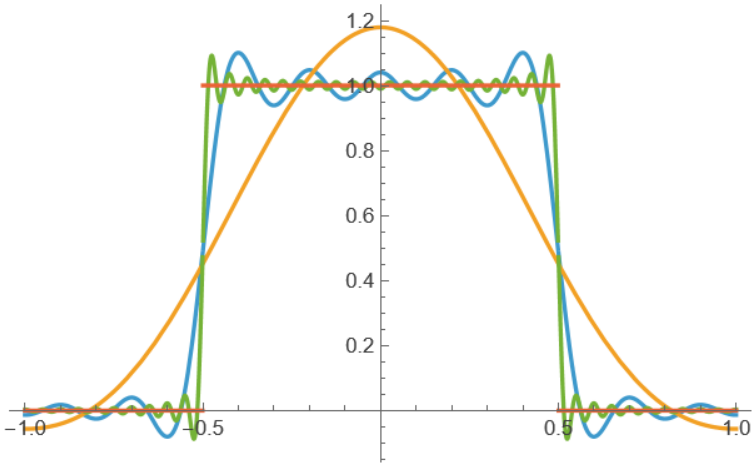

Figure 6. blue one is 5 Hz, orange one is 1 Hz , green one is 20 Hz and the red one is the main signal.

As seen from the figure, if f_0 is larger, we are closer to the main signal we had. this is because increasing the f_0 leads to bigger range of present frequencies in the filtered signal.

Now we check the Parseval's Theorem for the $f(t)$. we should check if this equation is true:

$$\int_{-\infty}^{\infty} |x(t)|^2 dt = \int_{-\infty}^{\infty} |X(f)|^2 df$$

we do this using *Equal* function in Mathematica.

```
Equal[Integrate[Abs[F[f]]^2, {f, -Infinity, Infinity}], Integrate[Abs[f[t]]^2, {t,-Infinity,
    Infinity}]]
```

and the output is "True". which means everything worked fine.

Now, we want to take the fourier inverse of $2F(2f)$. As we know from the time scaling property, the output of this inverse fourier transform will be $f(\frac{t}{2})$. We check this in Mathematica using these lines of code.

```
f0 = 200;
finverse[t_] = Integrate[{2 F[2 f]*Exp[I*2 Pi*f*t]}, {f, -f0, f0}];
Plot[finverse[t], {t, -10, 10}, Exclusions -> None]
```

The graph of this inverse fourier transform is as follows.

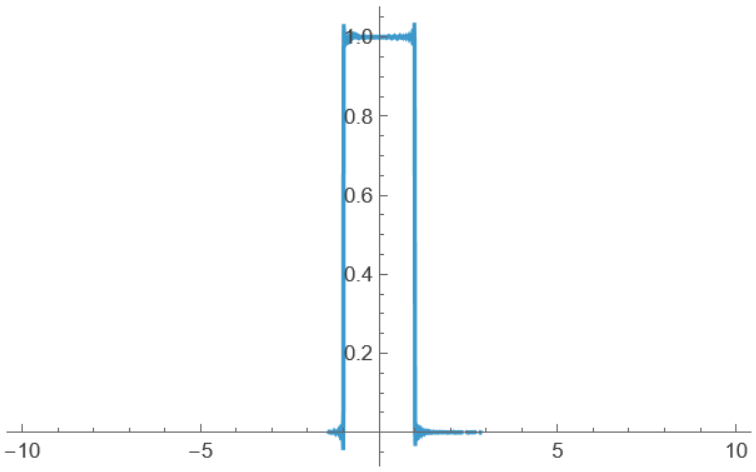

Figure 7. The inverse fourier transform

It's obvious that this signal is $f(\frac{t}{2})$.

4 Problem 3: (Analyzing aliasing effect on images in MATLAB)

this problem's objective is to simulate the Moire effect in MATLAB. first we use these lines of code to generate an image with periodic pattern.

```
N = 512;
[X,Y] = meshgrid(linspace(-1,1,N),linspace(-1,1,N));
R_sq = X.^2 + Y.^2;
original_img =cos(150 * R_sq);
imshow(original_img);
title('Original Signal (512x512)');
```

The image is like this.

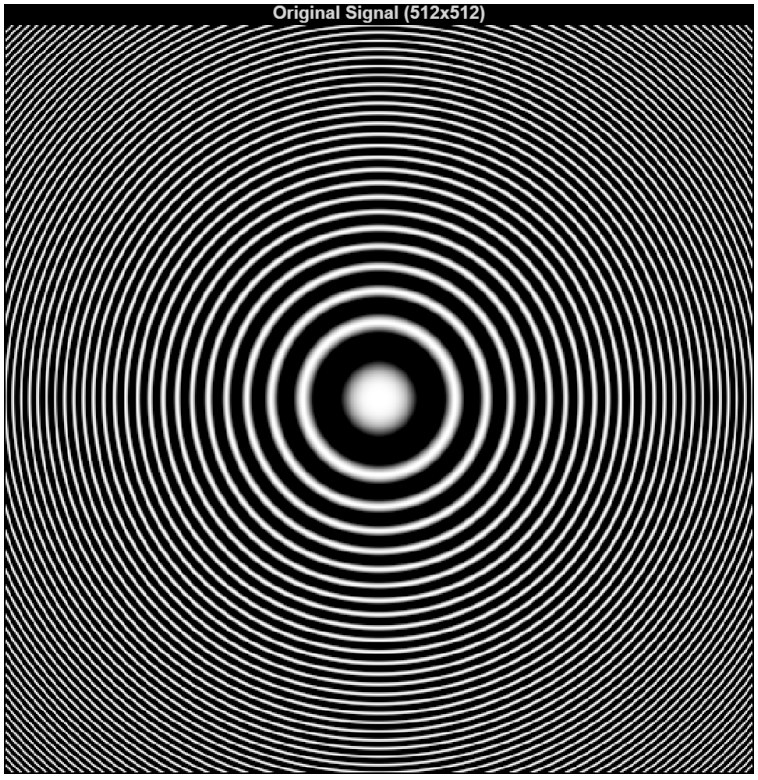

Figure 8. image with a periodic pattern.

we want to downsample this image. we do this using these lines of code.

```
downsample_factor = 5;
resized_img = original_img(1:downsample_factor:end, 1:downsample_factor:end);
imshow(resized_img);
```

the output is this image.

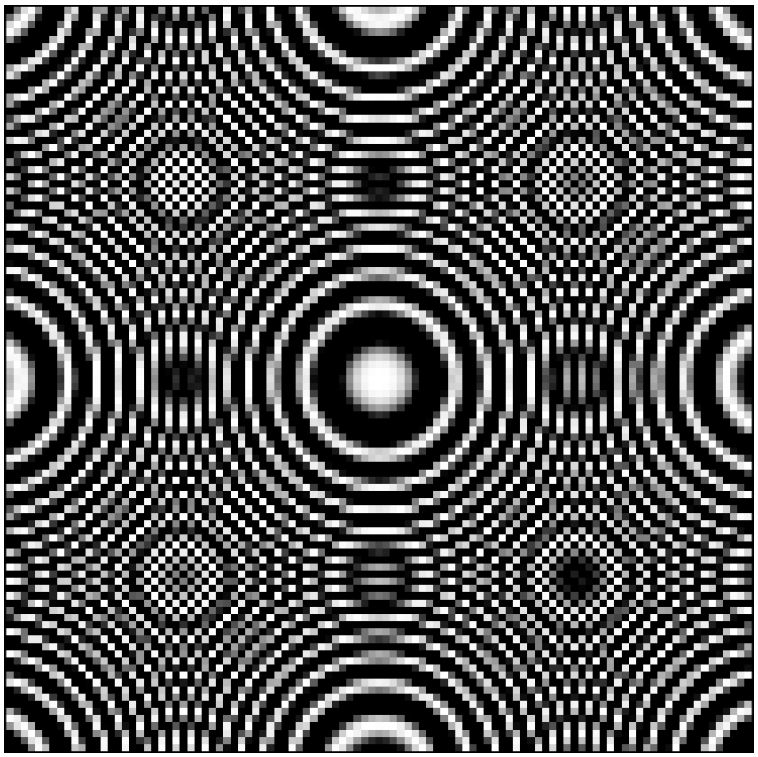

Figure 9. downsampled image

What we've done is that we just downsampled our image by a factor of 5 in both directions. It means we took the first pixel and then the sixth pixel and then the eleventh pixel and so on. Our original image had some high frequencies at the edges and when we downsampled the image, we reduced the sample rate therefore, we exceeded the Nyquist limit. When this happens, aliasing happens and we see a moire pattern. those artificial circles that werent present in the main image, are moire patterns.

Now, we should present a method that prevents aliasing. we use the Gaussian filter. It's implementation is as follows.

```
sigma = 3;
filtered_img = imgaussfilt(original_img, sigma);
resized_img_aa = filtered_img(1:downsample_factor:end, 1:downsample_factor:end);
imshow(resized_img_aa,[]);
```

The Gaussian filter is a low-pass filter that will blur the images and remove its high frequencies. It works by getting a weighted average of the pixels in a neighborhood, where the weights are calculated through the 2D Gaussian distribution. If σ gets bigger, the Gaussian will spread out more. therefore, more *sigma* leads to stronger blurring. we know that the Fourier transform of a Gaussian is another Gaussian but with an inverse relationship between its σ and its width. therefore the bigger the σ is, the narrower the signal is in the frequency domain. we find a decent σ for this case by testing some of them. I choose $\sigma = 3$. The output image is as follows.

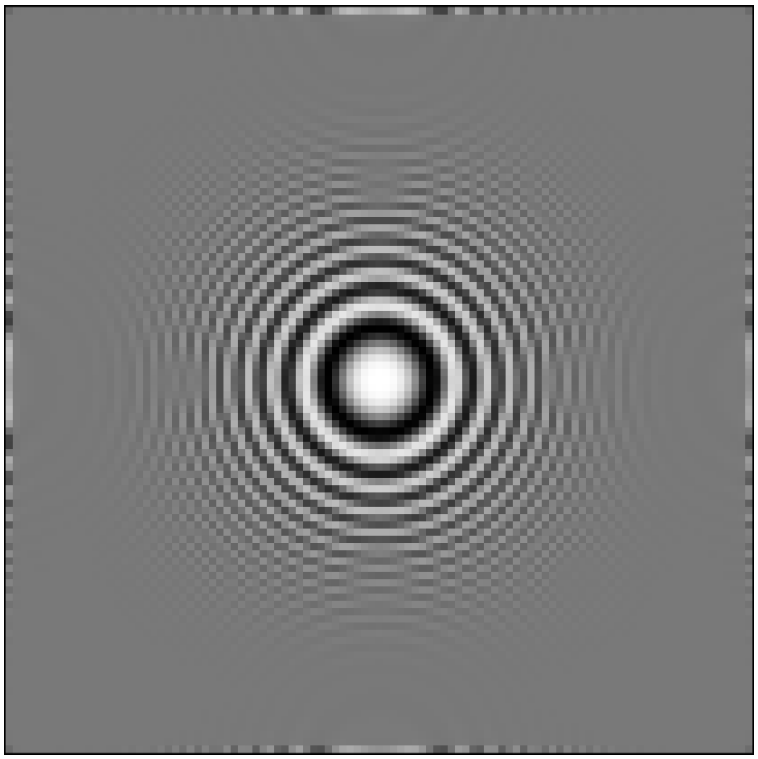

Figure 10. downsampling after the Gaussian filter

This Gaussian filter has some advantages. It is computationally efficient, meaning that you can apply 1D Gaussian filters to each dimension and have the same result. If the background noise is normal, then the Gaussian filter will automatically reduce that noise. The disadvantages are reducing image sharpness and generating bad results for extremely white and black values that were caused by impulse. We can use this method for anti-aliasing before the downsampling. and we cannot use this method when we want to preserve the sharp edges or work with noises that have impulses. even when we have a strict frequency band isolation we shouldn't use this filter.

5 Problem 4: (Sampling Theory and reconstruction of signals using Mathematica)

In this problem we want to sample e^{-t^2} signal and find the Fourier transform of the samples and then using the inverse Fourier transform, reconstruct the signal. The codes are like this:

```
Ts = 0.1;
nmax = 60;
x[t_] := Exp[-t^2];
Plot[x[t], {t, -3, 3}]
xSampled[t_] := Sum[x[n*Ts]*DiracDelta[t - n*Ts], {n, -nmax, nmax}];
X[f_] = FourierTransform[xSampled[t] *Ts, t, f, FourierParameters -> {0, -2*Pi}];
Plot[{Re[X[f]]}, {f, -2, 2}, Exclusions -> None]
f2 = 3;
xFiltered2[t_] = Integrate[X[f]*Exp[I*2 Pi*f*t], {f, -f2, f2}];
Plot[xFiltered2[t], {t, -3, 3}]
```

in this case the B is equal to 6 Hz. therefore we should have $T_s \leq 0.1667$. Thus, everything should work fine. The three images generated by the code are as follows.

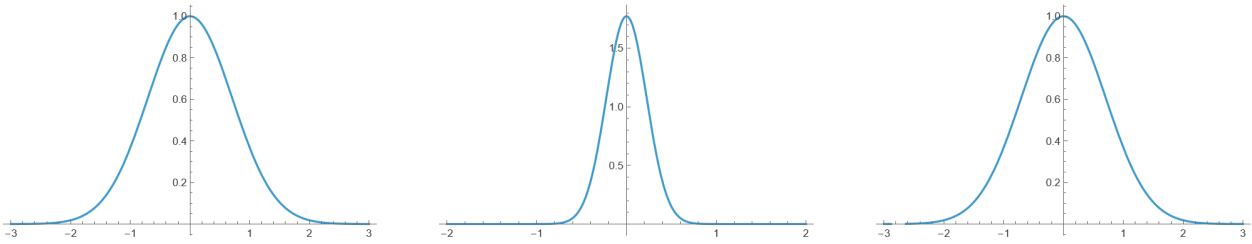

Figure 11. The left image is the e^{-t^2} signal. The middle image is the fourier transform of its samples. the right image is the inverse fourier transform which is similar to the main signal.

Now if we change the T_s to a number outside the range like 0.3, the output will be like this:

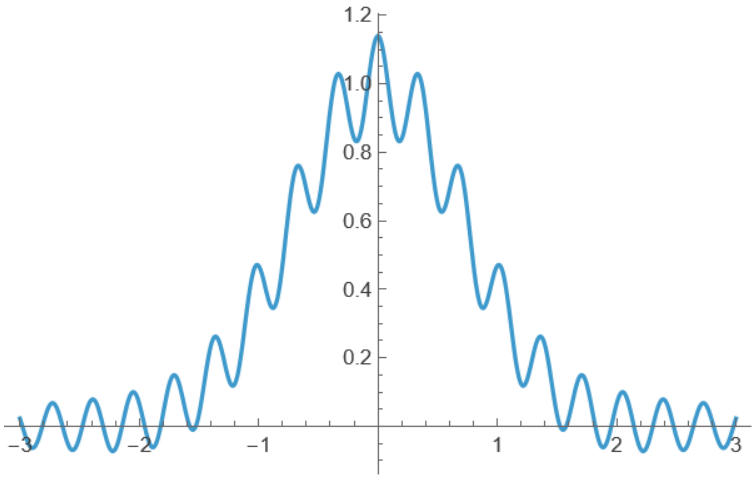

Figure 12. The Moire effect is obviously present here.

6 Conclusion

In order to filter a signal you can just take the Fourier Transform and then use the Inverse Fourier Transform Integral in the desired bandwidth. Some times doing so leads to aliasing, like when you sample the signals first and then you take the Fourier transform of them. In these situations you should be aware of the Nyquist-Shannon theorem. A simple method that we practiced during the project was Gaussian filter. It's one of the typical ways of dealing with aliasing.